

MEDIATEK

MT8676 Android Suspend and Resume User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	郑孝俊	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 Suspend/Resume 4

 1.1 概述 4

 1.2 架构/流程概览 4

 1.2.1 Android Suspend/Resume 流程 4

 1.2.2 Kernel Suspend/Resume 流程 6

 1.3 配置/客制化指南 7

 1.4 常见问题/故障排除 7

 1.4.1 如何判定系统休眠成功 7

 1.4.2 如何确认唤醒源 7

 1.4.3 唤醒源梳理 8

 1.4.4 不能休眠问题如何分析 9

 1.4.5 如何分析休眠模式下功耗大的问题 10

附件一 附加条款 11

图片目录

图 1-1. Android Suspend/Resume 流程 4

图 1-2. Android Suspend/Resume 源代码 5

图 1-3. Kernel Suspend/Resume 流程 6

图 1-4. Wakelock dump 示意图 10

表格目录

表 1-1. MT8676 唤醒源列表 8

1 Suspend/Resume

1.1 概述

本章节主要介绍 MT8676 Suspend/Resume 以及常见问题的处理方法。

在 MT8676 平台上, Suspend 即 suspend to RAM, 此状态使所有的设备进入低功耗状态, 仅保留 RAM 自刷新。

Suspend Power State: 整个系统待机, 耗电非常低。

- Tasks “freezed”
- 外部设备/内部部分模块关电或者进入低功耗模式
- System PLL/clock close
- ARM off
- DRAM 自刷新
- PMIC 进入低功耗模式
- VCORE off
- SPM run
- 等待硬件唤醒事件

1.2 架构/流程概览

1.2.1 Android Suspend/Resume 流程

Android 系统 Suspend 主要分为两个阶段, 第一阶段是灭屏, 即关闭显示, 进入浅睡眠。系统进入浅睡眠后如果没有 wakelock, 就会触发 Kernel Suspend 流程。Resume 过程相反。参考图 1-1:

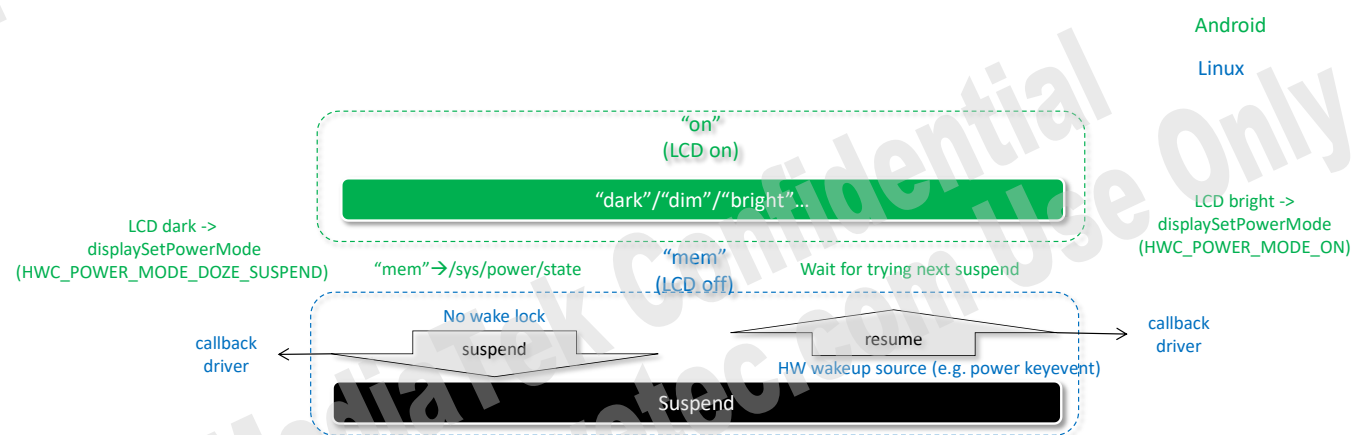


图 1-1. Android Suspend/Resume 流程

对应的源代码如下:

```
void SystemSuspend::initAutosuspendLocked() {
    if (mAutosuspendThreadCreated) {
        LOG(INFO) << "Autosuspend thread already started.";
        return;
    }

    std::thread autosuspendThread([this] {
        auto autosuspendLock = std::unique_lock(mAutosuspendLock);
        bool shouldSleep = true;

        while (true) {
            if (!mAutosuspendEnabled) {
                mAutosuspendThreadCreated = false;
                return;
            }
            // If we got here by a failed write to /sys/power/wakeup_count; don't sleep
            // since we didn't attempt to suspend on the last cycle of this loop.
            if (shouldSleep) {
                mAutosuspendCondVar.wait_for(autosuspendLock, mSleepTime,
                    [this] { return !mAutosuspendEnabled; });
            }

            if (!mAutosuspendEnabled) continue;

            string wakeupCount;
            {
                autosuspendLock.unlock();

                lseek(mWakeupCountFd, 0, SEEK_SET);
                wakeupCount = readFd(mWakeupCountFd);

                autosuspendLock.lock();
            }

            if (wakeupCount.empty()) {
                PLOG(ERROR) << "error reading from /sys/power/wakeup_count";
                continue;
            }

            shouldSleep = false;

            mAutosuspendCondVar.wait(
                autosuspendLock, [this] { return mSuspendCounter == 0 || !mAutosuspendEnabled; });
            // The mutex is locked and *MUST* remain locked until we write to /sys/power/state.
            // Otherwise, a WakeLock might be acquired after we check mSuspendCounter and before we
            // write to /sys/power/state.
            if (!mAutosuspendEnabled) continue;
            if (!mHasAliveAutosuspendTokenLocked()) {
                disableAutosuspendLocked();
                continue;
            }
            if (!WriteStringToFd(wakeupCount, mWakeupCountFd)) {
                PLOG(VERBOSE) << "error writing from /sys/power/wakeup_count";
                continue;
            }
            bool success = WriteStringToFd(kSleepState, mStateFd);
            shouldSleep = true;

            {
                autosuspendLock.unlock();

                if (!success) {
                    PLOG(VERBOSE) << "error writing to /sys/power/state";
                }

                struct SuspendTime suspendTime = readSuspendTime(mSuspendTimeFd);
                updateSleepTime(success, suspendTime);

                std::vector<std::string> wakeupReasons = readWakeupReasons(mWakeupReasonsFd);
                if (wakeupReasons == std::vector<std::string>({kUnknownWakeup})) {
                    LOG(INFO) << "Unknown/empty wakeup reason. Re-opening wakeup_reason file.";
                }
                mWakeupReasonsFd =
                    std::move(reopenFileUsingFd(mWakeupReasonsFd.get(), O_CLOEXEC | O_RDONLY));
            }
            mWakeupList.update(wakeupReasons);
            mControlService->notifyWakeup(success, wakeupReasons);

            // Take the lock before returning to the start of the loop
            autosuspendLock.lock();
        }
    });
    autosuspendThread.detach();
    mAutosuspendThreadCreated = true;
    LOG(INFO) << "automatic system suspend enabled";
}
```

图 1-2. Android Suspend/Resume 源代码

主要的流程说明:

1. Sleep mSleepTime: 初始值为 100ms, 会随着 Suspend 执行成功与否而变更。
2. 当读取 /sys/power/wakeup_count 值时, 如果此时底层有 activewakeup source, 读取操作将会在 Kernel 中被阻塞。
3. 直到没有 active wakeup source 时, 会返回 wakeup_count。
4. 写回 wakeup_count, 如果写回成功则继续执行。
5. 将 mem 写入 /sys/power/state 后, 系统将进入 Kernel Suspend 流程。如果 Suspend 成功, 程序将在该行停留, 等待 Kernel Resume 继续执行。如果 Suspend 失败, 则 mSleepTime 将会加倍, 并重新循环尝试 Suspend。

1.2.2 Kernel Suspend/Resume 流程

关于 Kernel Suspend/Resume 的流程，可参考图 1-3：

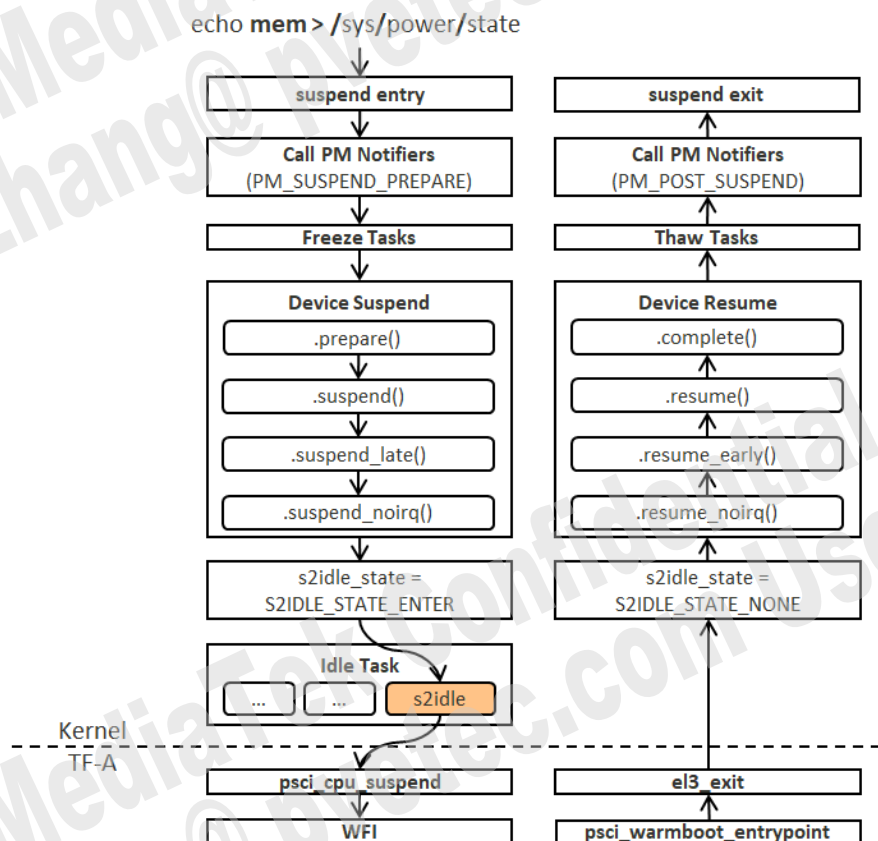


图 1-3. Kernel Suspend/Resume 流程

上层将 mem 写入 /sys/power/state，触发 Kernel Suspend 流程：

1. 系统首先 freeze process，先冻结用户空间进程（userspace process），接着冻结内核线程（kernel thread）。如果这一步发现有 wakelock，则会中止 Suspend 流程。若冻结操作成功，则 Suspend 过程将继续进行。
2. 在成功冻结进程之后，系统会开始运行 device suspend flow。按照执行顺序分别调用 device 注册的 prepare/suspend/suspend_late callback，完成各个 device driver 在休眠阶段需要做的准备工作。
3. 进入 s2idle_enter()，s2idle_state = S2IDLE_STATE_ENTER：所有 CPU 进入 idle loop。
4. 最后进入 TF-A psci_cpu_suspend。
5. 关闭不使用的硬件，整个系统进入低功耗休眠模式。

进入休眠模式之后，如果有唤醒源唤醒请求，系统会被唤醒，进入 Resume 流程，Resume 过程与 Suspend 相反。

1.3 配置/客制化指南

由于 Suspend/Resume 流程大部分是 Android 和 Kernel 的原生流程，因此不太需要客制化的设定。

1.4 常见问题/故障排除

1.4.1 如何判定系统休眠成功

屏幕熄灭可能仅表示系统进入浅睡眠，这并不代表系统已经成功进入休眠状态。

判定系统是否休眠成功需要查看 Kernel 日志，若系统成功进入 Suspend，则 Kernel 日志将不再打印新信息。

如果需要通过日志判断休眠是否成功，需要先下这几行命令打开更多的调试日志信息：

```
adb shell "echo 8 8 8 8 > /proc/sys/kernel/printk"
adb shell "echo 1 > /sys/module/kernel/parameters/initcall_debug"
adb shell "echo 1 > /proc/mtprintk"
```

Kernel 日志中，关键字“PM: Syncing filesystems ...”表示 Kernel 开始走 Suspend 流程。

Kernel 日志中，关键字“suspend of devices complete after xxx msecs”表示 device suspend 完成。

Kernel 日志中，关键字“late suspend of devices complete after xxx msecs”表示 device late suspend 完成。

Kernel 日志中，关键字“noirq suspend of devices complete after xxx msecs”表示 device noirq suspend 完成。

Kernel 日志中，关键字“suspend enter”表示 suspend 流程结束，系统已进入 suspend 状态。

1.4.2 如何确认唤醒源

在 Kernel 日志中搜索关键字“suspend wake up by”，可以查看唤醒源。

如：

Pwrkey 唤醒（及其他 EINT 唤醒）：

```
[SPM] suspend wake up by R12_EINT_EVENT_B, timer_out = 207308
```

如果要看具体是由哪个 EINT 唤醒，需先从日志查看：

```
EINT xxx is pending
```

再通过 `cat /proc/interrupts` 看 xxx 对应的是哪个中断。

Modem 相关唤醒:

[SPM] suspend wake up by R12_CCIF0_EVENT_B, timer_out = 1825253

定时器 (PCM_Timer) 唤醒:

[SPM] suspend wake up by PCM_TIMER, timer_out = 65612

1.4.3 唤醒源梳理

目前 MT8676 支持的唤醒源列表如下:

表 1-1. MT8676 唤醒源列表

Name	Control Bit	Description	Can it be Disabled
R12_PCM_TIMER	0	设定计时器, 超时后唤醒系统	Y
R12_KP_IRQ_B	2	检测到键盘按下/释放动作	Y
R12_APWDT_EVENT_B	3	RGU 在 suspend mode 中唤醒 CPU	Y
R12_APXGPT1_EVENT_B	4	AP GPT 计时器超时事件	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC (Wi-Fi/BT/GPS) 唤醒系统	Y
R12_EINT_EVENT_B	6	EINT 事件唤醒系统	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout 唤醒系统	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF 唤醒事件	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF 唤醒事件	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM 相关唤醒系统	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor 相关唤醒系统	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP 相关唤醒系统	Y
R12_USBX_CDSC_B	14	USB 相关唤醒系统	Y
R12_USBX_POWERDOWN_B	15	USB 远程唤醒	Y
R12_SYS_TIMER_EVENT_B	18	System timer 唤醒系统	N
R12_EINT_EVENT_SECURE_B	19	EINT event 唤醒系统	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ 唤醒事件	Y
R12_MD2AP_PEER_EVENT_B	23	MD 相关唤醒系统	Y
R12_MD1_WDT_B	25	MD1 看门狗超时	Y

Name	Control Bit	Description	Can it be Disabled
R12_REG_CPU_WAKEUP_B	28	内部唤醒源	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS 的唤醒源	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe 相关唤醒源	Y
R12_MSDC_WAKEUP_EVENT_B	31	MSDC 相关唤醒源	Y

您可以按照以下方式禁用列表中可以禁用的唤醒源。但是，请注意，只有在“Can it be Disabled”列中标注为可禁用（Y）的唤醒源才能被禁用：

使用系统提供的调试命令可以快速禁用特定的唤醒源：

1. 使用如下命令读取当前系统唤醒源设定：

```
cat /proc/mtk_lpm/power/suspend_ctrl | grep -i wake_src
```

2. 在表 1-1 的“Control Bit”栏位中找到您想要禁用唤醒源所对应的的 control bit 位，然后将读出的 wake_src 的对应 bit 写 0，最后用下述命令将修改后的 wake_src 写进系统（不需要禁用的唤醒源的 control bit 位不进行修改）：

```
echo wake_src 0x***** > /proc/mtk_lpm/power/suspend_ctrl
```

3. 如果想要再启用某个唤醒源，就将对应的 control bit 再写 1 即可。

1.4.4 不能休眠问题如何分析

如系统灭屏后无法进入休眠，可以用以下命令判定是哪个 wakelock 阻止系统进入休眠：

- 查看用户空间（userspace）有哪些模块持锁。

dumpsys power

如

```
Wake Locks: size=1
  SCREEN_DIM_WAKE_LOCK          'CAR.POWER' ACQ=-5m14s440ms (uid=1000 pid=1467)

Suspend Blockers: size=4
  PowerManagerService.WakeLocks: ref count=1
  PowerManagerService.Display: ref count=1
  PowerManagerService.Broadcasts: ref count=0
  PowerManagerService.WirelessChargerDetector: ref count=0

Display Power: state=ON
```

- 查看 Kernel 中有哪些模块持锁。

```
cat /sys/kernel/debug/wakeup_sources
```

观察输出结果的第 5 列 active_since，数字不为 0 且一直在增大的就是阻止系统进入待机的 wakelock。

例如图 1-4 中就是 USB 阻止了系统进入待机状态：

```
spm3675p1_64:/ # cat /sys/kernel/debug/wakeup_sources
```

name	active_count	event_count	wakeup_count	expire_count	active_since	total_time	max_time	last_change	prevent_suspend_time
event2	0	0	0	0	0	0	0	0	0
event1	3	3	0	0	0	203	145	41989	0
event3	0	0	0	0	0	0	0	0	0
event0	0	0	0	0	0	0	0	0	0
inotify	20	20	0	0	0	4	1	130596	0
eventpoll	2	2	0	0	0	0	0	31562	0
[timerfd]	30	30	0	0	0	0	0	130596	0
[timerfd]	18	18	9	0	0	1223	151	41937	0
[timerfd]	0	0	0	0	0	0	0	0	0
[timerfd]	18	18	4	0	0	575	151	121306	0
[timerfd]	20	20	2	0	0	284	131	37344	0
[timerfd]	0	0	0	0	0	0	0	0	0
[timerfd]	0	0	0	0	0	0	0	0	0
eventpoll	44	56	0	0	0	0	0	121306	0
gspwakeup	0	0	0	0	0	0	0	0	0
bt_drv	0	0	0	0	0	0	0	0	0
usbifmCtrl	3	3	0	0	0	2872	1456	43697	0
MT662x	7	7	0	0	0	374	72	52259	0
rpm1	0	0	0	0	0	0	0	0	0
rpm0	0	0	0	0	0	0	0	0	0
usb-hdrc	2	2	0	0	107302	134244	107302	41949	0
mdla	0	0	0	0	0	0	0	0	0
slbc	0	0	0	0	0	0	0	0	0

图 1-4. Wakelock dump 示意图

该命令需要在 UART 下输入，因为插入 USB 本身会阻止系统进入待机状态。

1.4.5 如何分析休眠模式下功耗大的问题

- 首先看是否有频繁唤醒问题，如果有，请参考 1.4.2 章节确认唤醒源。
- 如果成功进入休眠状态但功耗偏大，请先检查是否有未关闭的外设耗电。
- 如果定位到是 MediaTek SoC 造成的功耗偏大，请提供日志给 MediaTek 进行分析。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前所述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。